NetMind: Review 2 checklist

AI-Powered Network Intelligence and LLM Network Copilot

Course	Computer Networks Lab (BCSE308P), VIT Vellore
Team	Mohammed Affan (24BAI0014), Hardik (24BCI0017)
Repository	github.com/mo-affan/NetMind
Prepared	15 October 2026

How to use this: go through the checklist on page 1 the evening before and again 30 minutes before the review. Sections 3 to 8 are what you present; section 9 is what to show live; section 10 is the viva preparation; Appendix A lists every command and Appendix B shows how to start NS2 cleanly. Every number in this document is real output from NetMind or from our NS2 run; where data is synthetic, it says so, and you should say so too.

1. Before the review: checklist

Laptop

- ☐ **git pull** in C:\Users\moham\NetMind, then **python -m pip install -r requirements.txt** (Streamlit 1.51+ for dark mode)
- ☐ **python -m pytest -q** passes (note the pass count to quote it)
- ☐ **python -m netmind demo** runs and shows 3 incidents
- ☐ **python -m netmind dashboard** opens at http://localhost:8501 (keep it running in a second window)
- ☐ Dashboard: a **Synthetic demo** run and the **real NS2 trace** (ns2\results\netmind.tr) are both loaded as runs, so you can switch between them without waiting
- ☐ Copilot answers "Why is the network slow?" on the Overview page
- ☐ Ubuntu: **~/netmind_ns2** has the NS2 output (netmind.tr, netmind.nam); **python3 check_trace.py netmind.tr** shows 5/5 PASS
- ☐ NAM opens (export TCL_LIBRARY / TK_LIBRARY if needed); if not, screenshots or the dashboard instead
- ☐ Charger, screen zoom set so terminal text is readable, other apps closed, notifications off

Material

- ☐ Review 2 slides (architecture, modules, results, NS2, demo, next steps)
- ☐ Review 1 proposal and the faculty feedback (NS2 added) to show it was acted on
- ☐ Screenshots as backup: dashboard Overview, Incidents, Topology with a failed link, Copilot, terminal demo, NS2 check output, NAM at 3 s / 7 s / 8.5 s
- ☐ Who presents which part is agreed (section 3)

2. The 30-second opening

> “Network monitoring tools show numbers like latency and packet loss but leave it to the user to work out what is wrong. NetMind measures the network from its traffic, detects unusual behaviour with an Isolation Forest model, and explains it in plain language through a Copilot that combines the measurements with a networking knowledge base. We use three kinds of input: real captures, an NS2 simulation with congestion and a link failure at known times, and a synthetic demo with three known problems, so we can check that what NetMind finds matches the truth.”

Problem and objectives

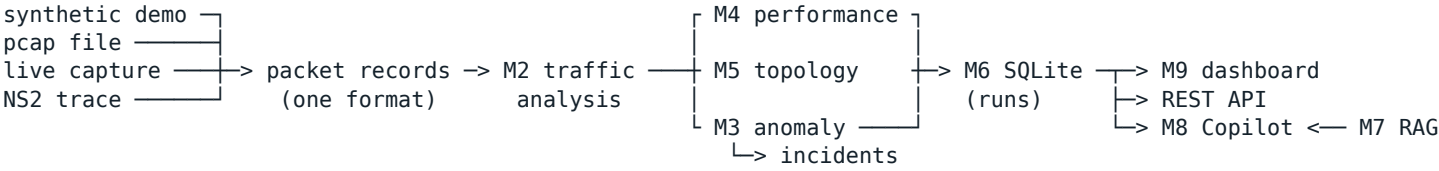
- **Problem:** raw metrics (RTT, loss, connection counts) need expertise to interpret; problems such as congestion, scans or floods are noticed late.
- **Objective 1, measure:** protocols, flows, latency, throughput, loss and jitter from traffic.
- **Objective 2, detect:** flag unusual time windows and group them into typed incidents.
- **Objective 3, explain:** a Copilot that answers questions using the run's data and retrieved networking knowledge (RAG), with sources. Advisory only; it never changes the network.
- **Objective 4, routing:** topology graph, Dijkstra, Link State vs Distance Vector, link failure.
- **Objective 5, simulate:** NS2 scenario with congestion and rerouting to validate the analysis.

3. Who presents what

Part	Modules	Presenter
Input and traffic analysis	M1 capture (Scapy, pcap, synthetic), M2 protocols and flows	Hardik
Topology and routing, storage	M5 graph, Dijkstra, LS, DV; M6 SQLite	Hardik
Dashboard	M9 Streamlit dashboard (10 pages)	Hardik (lead)
Performance and anomaly detection	M4 metrics; M3 Isolation Forest, deviation check, incidents	Affan
Knowledge and Copilot	M7 RAG; M8 Copilot (offline, Claude, Ollama)	Affan
NS2 simulation	TCL scenario, trace parser, checker, lab run	Affan

Adjust if you split it differently; the owners match the README.

4. Architecture



- **Key design rule:** every input becomes the same packet-record table (time, addresses, ports, protocol, length, TCP flags/seq/ack, ICMP type/id/seq, DNS id). Every later module works on any source.
- **Pipeline** (netmind/pipeline.py) runs the modules in order and saves each analysis as a **run**.
- **Interfaces:** command line (python -m netmind ...), Streamlit dashboard, FastAPI REST API (/health, /runs, /ask, /path, /routing ...).
- **Stack:** Python, Scapy, pandas, NumPy, scikit-learn, NetworkX, SQLite, Streamlit, Plotly, FastAPI, NS2.

5. Modules: what each does and how

Module	Folder	What it does and how
M1 Input	`netmind/capture/`	Scapy live capture (sniff, needs Npcap and admin) and pcap files (rdpcap); NS2 wired trace parser (event time from to type size flags fid src dst seq id); synthetic demo: 3 clients browsing (DNS, TCP handshake, HTTP/HTTPS, FIN) and pinging the gateway, with congestion 70-100 s, port scan 120-128 s from 192.168.1.66 and ping flood 150-158 s injected. Deterministic (seed 7), labelled synthetic.
M2 Traffic	`netmind/analyzer/`	Protocol from the IP header (TCP/UDP/ICMP) and well-known ports (53 DNS, 80 HTTP, 443 HTTPS). **Flow** = 5-tuple (src IP, src port, dst IP, dst port, protocol), both directions merged. Outputs protocol shares, flows, top talkers.
M4 Metrics	`netmind/performance/`	Per 2 s window: **latency** = SYN to SYN-ACK (ack = seq+1), ICMP echo request to reply (same id, seq), DNS query to response (same id), averaged per path; **throughput** = bytes × 8 / window; **loss** = (TCP retransmissions + unanswered pings) / (TCP data segments + pings); **jitter** = mean $\lvert RTT_i - RTT_{i-1} \rvert$ on the same path; packets/s; utilisation.
M5 Topology	`netmind/topology/`	NetworkX graph of hosts that exchanged packets; weights = median RTT or hops. **Dijkstra** shortest path. **Link State** (OSPF idea): each node knows the map, runs Dijkstra. **Distance Vector** (RIP idea, Bellman-Ford): neighbours exchange tables each round until nothing changes; rounds are reported. LS and DV give identical costs (tested); DV reroutes after a link failure.
M6 Storage	`netmind/storage/`	SQLite (data/netmind.db): runs table plus per-run metrics, anomalies, incidents, nodes, links, flows, protocols, RTT samples.
M3 Anomaly	`netmind/anomaly/`	10 features per window (packets/s, avg packet size, connections, connection duration, distinct destination ports, TCP/UDP ratio, bytes/s, latency, loss, retransmissions). Baseline = first 60 s of the demo (default 30%, at least 8 windows = 16 s; less → detection skipped, never "normal"). **Isolation Forest** (200 trees), score on 0-1 scale (≈0.5 normal, near 1 unusual); threshold = 99th percentile of baseline + 0.03 + 0.25/√n. **Deviation check**: robust $z = (x - \text{median}) / (1.4826 \times \text{MAD})$. Flag if $z \geq 6$, or forest score > threshold and $z \geq 4$. Consecutive flags → **incidents**, typed as congestion, port scan, flood, traffic spike, reroute.
M7 RAG	`netmind/rag/`	9 networking documents written for the project → 36 chunks (by section, ~140 words). TF-IDF (1-2 grams) by default, works offline; Sentence Transformers optional; cosine search (FAISS optional); top 4 chunks go into the prompt.
M8 Copilot	`netmind/copilot/`	Prompt = question + network state + incidents with evidence + retrieved knowledge [1]-[4]. Backends: offline rule-based (default, deterministic), Claude API, local Ollama; falls back to offline if the LLM is unreachable. Cites sources. Advisory only.
M9 Dashboard	`netmind/dashboard/`	Streamlit, 10 pages (section 8), light and dark themes; also the CLI and the REST API.
NS2	`ns2/`	netmind_scenario.tcl, trace parser (parse_trace.py), standard-library checker (check_trace.py), lab guide (docs/NS2_LAB_EXECUTION.md). Details in section 7.

6. Results to quote

Synthetic demo (python -m netmind demo, 180 s, seed 7: generated traffic, not a capture)

Injected event	Time	Detected as	Evidence
Congestion	70-100 s	Network congestion	Latency 146 ms vs 26.5 ms normal (5.5×); loss about 9.5% vs 0
Port scan	120-128 s	Possible port scan, from 192.168.1.66	311 destination ports per window vs about 10
Ping flood	150-158 s	Traffic flood, ICMP from 192.168.1.66	551 packets/s vs 41; average packet 104 vs 660 bytes

- 13,511 packets, 9 hosts, 1,731 flows; 90 windows, 23 flagged (all inside the three events).
- **Health score** 82.4/100 (Fair) for the demo, worst window 0 at 76 s (congestion); normal-only traffic scores 99.5 (Good).
- **TCP connections:** 1,363 attempts, 363 established, 1,000 refused, all from 192.168.1.66 (flagged as a likely scanner). **DNS:** 364 lookups, 4 unanswered, median 29.3 ms.
- All three events found, with the right type, time and source.
- **False alarms** on normal-only synthetic traffic (5 lengths × 10 seeds, about 1,350 windows): 3 windows flagged, under 0.5%. Before the tuning it was 18.

Tests

- Automated pytest suite: capture, metrics, topology (LS = DV, DV reroutes), anomaly detection, NS2 parser, pipeline, RAG, Copilot, CLI, edge cases (empty and very short captures), dashboard logic and every dashboard page (Streamlit AppTest).
- On the laptop before the dashboard redesign: **61 passed, 2 skipped**. After git pull, run it again and quote the new number.

7. NS2 simulation

Status	State
Implemented	Done: TCL scenario, trace parser, checker reviewed and fixed
Validated	Done: parser and NetMind pipeline tested on a sample trace
Executed	**Done on 15 Oct 2026**: scenario run in NS2 on Ubuntu 24.04 (on the laptop); all 5 checks passed on the real trace; re-run on 28 Oct with the same result

Scenario (5 nodes)

Time	Event	Expected
0.5 s	TCP (FTP) n0 → n3 starts, window 6 packets	No drops, normal delay
3.0-6.0 s	UDP CBR 0.9 Mb/s n1 → n3 on the 1 Mb n2-n3 bottleneck (queue 10)	Queue fills: drops, delay rises (congestion)
7.0 s	Link n2-n3 goes down	Distance Vector reroutes TCP via n4
8.5 s	Link n2-n3 back up	TCP returns to n2-n3
10 s	End	netmind.tr (trace) and netmind.nam (animation)

Real NS2 results (check_trace.py on the real trace)

Check	Result
No drops before the UDP burst (0.5-3 s)	PASS: 0 drops
Drops during the UDP burst (3-6 s)	PASS: 21 drops
Delay higher during the burst	PASS: 43.0 ms → 103.1 ms
Traffic rerouted via n4 during the link failure (7-8.5 s)	PASS
TCP delay higher on the backup path	PASS: 115.2 ms

DropTail vs RED queue (compare_traces.py, executed 28 Oct)

During the UDP burst (3-6 s)	DropTail	RED
Mean delay	103.1 ms	**71.3 ms (31% lower)**
Drops (loss)	21 (5.24%)	25 (6.22%)
TCP drops	8	10
TCP goodput	0.153 Mb/s	0.150 Mb/s

Normal and link-failure phases were identical for both. RED drops early at random once the average queue passes 3 packets, so the queue stays short: less delay, slightly more loss.

- **Definitions** used for NS2: delay = receive time − send time (one-way, NS2 knows both); loss = dropped / (delivered + dropped), packets still in flight excluded; jitter per flow; a retransmission = same flow id and sequence number sent again.
- **Why NS2:** problems at known times in a repeatable network, so we can check NetMind's measurements and detection against the truth, and see Distance Vector rerouting.
- **Show:** the check output, NAM at 3 s / 7 s / 8.5 s, and the trace analysed in the dashboard (Incidents: congestion and link failure; Topology: fail n2-n3).

8. Dashboard pages

Page	What to point out
Overview	Whole run as one traffic chart with incidents as red bands; verdict ("3 incidents in 180 s"); six headline numbers with trend lines; latency and loss vs normal; protocols; top talkers; Copilot panel
Incidents	Anomaly score over time; per incident: severity, what stood out vs normal, zoomed chart, "Ask the Copilot about this incident"
Traffic	Packets and bytes by protocol; searchable flow table (try 192.168.1.66)
Hosts & services	Every host with scope, role and services; TCP connection outcomes (192.168.1.66 called out as a likely scanner: 1,000 refused attempts); DNS lookups and top domains
Performance	Health score, latency, jitter, loss, throughput, packets/s, retransmissions with the normal level; RTT by type
Topology & routing	Host graph, Dijkstra path, Link State vs Distance Vector table; **Simulate a link failure** shows which routes change and DV rounds
NS2 lab	The scenario topology and timeline, the TCL script, the run commands, the five checks on an uploaded trace (5 × PASS), and DropTail vs RED from two uploaded traces
Copilot	Chat with suggested and per-incident questions, sources, and the exact prompt given to the model
Compare runs	Two runs side by side (demo vs normal-only traffic)
Data & export	Markdown report and CSV downloads, raw tables, baseline, delete a run

Sidebar: **Start a new run** (demo with seed and length, normal-only traffic, upload a pcap or NS2 trace, live capture). Light/dark: the three-dot menu at the top right → Settings.

9. Live demo script (about 5 minutes)

Time	Do	Say
0:00	Terminal: python -m netmind demo	Synthetic traffic with three problems injected at known times; labelled synthetic, so we know the truth.
0:30	Point at the window table and incidents	Every 2 s we measure rate, throughput, latency, jitter, loss. The detector learned normal from the first 60 s and found three incidents, each with evidence.
1:15	Dashboard Overview	Red bands are the incidents; worst latency is 5.5× normal.
1:45	Copilot panel: "Is anyone attacking my network?"	It names the scan, the attacker and what to check, and cites its sources. Advisory only.
2:30	Incidents page; Ask about the port scan	Severity, features that stood out vs normal, zoomed chart.
3:00	Topology & routing: fail a link	Dijkstra path; LS and DV agree; after the failure DV recomputes and the table shows what changed.
3:45	Ubuntu: check_trace.py output, NAM	Real NS2 run: congestion at 3 s, link failure at 7 s, reroute via n4. All five checks pass.
4:30	Dashboard: switch to the NS2 run	Same pipeline on the NS2 trace: congestion and link-failure incidents.

If something goes wrong

Problem	Do
Dashboard will not open	Use the terminal demo and python -m netmind ask "..."; same results
Port 8501 busy	python -m netmind dashboard --port 8502
NAM does not open	Show screenshots, or the NS2 run in the dashboard (Incidents, Topology)
Old run shown	Pick it in the sidebar Run box, or Start a new run → Generate and analyse
Asked for a real capture	python -m netmind capture --duration 60 in an Administrator terminal (Npcap); short captures report "not enough traffic" honestly

10. Viva questions and answers

Is this a networks project or an AI project?

Networks first: capture, protocol analysis, RTT/loss/jitter measurement, flows, topology, Dijkstra, Link State, Distance Vector and NS2 are the core. ML and the LLM only interpret what those measure.

How do you measure latency without sending anything?

From request/reply pairs already in the traffic: TCP SYN and SYN-ACK (ack = seq + 1), ICMP echo request and reply (same id and sequence), DNS query and response (same transaction id).

How do you know packets were lost if you never saw them?

Inferred: TCP resends lost data with the same sequence number, and a ping without a reply is lost. In NS2 the trace records drops directly (d events).

What is jitter, and why per path?

Variation in delay between consecutive packets. Different paths have different base delays; mixing them measures the difference between paths, not variation (it inflated jitter to about 20 ms).

What is a flow?

Packets sharing the 5-tuple (src IP, src port, dst IP, dst port, protocol); both directions merged.

Why Isolation Forest?

Unsupervised (no labelled attacks needed), fast; unusual windows are isolated in fewer random splits. Score = 2^(−average path length / c(n)): about 0.5 normal, near 1 unusual.

Why an extra deviation check?

The forest cannot split on a feature that never varied in training (loss is 0 in normal traffic), so it missed congestion. The robust z-score catches it and explains which features stood out.

How do you avoid false alarms?

$z \geq 4$ required behind a forest flag, a stricter threshold for short baselines, and a 16 s minimum baseline. Normal-only traffic: 3 of about 1,350 windows flagged (was 18).

Dijkstra vs Bellman-Ford?

Dijkstra takes the closest unfinished node each step and needs non-negative weights. Bellman-Ford relaxes all edges repeatedly, handles negative weights and is the basis of Distance Vector.

Link State vs Distance Vector?

LS (OSPF): routers flood link states, know the whole map and run Dijkstra. DV (RIP): routers only exchange distance tables with neighbours; simpler, slower to converge, can count to infinity.

What does NS2 add?

Problems at known times in a repeatable network, so NetMind's results can be checked against the truth, plus Distance Vector rerouting after a link failure.

What is RAG?

Retrieval-Augmented Generation: retrieve the most relevant knowledge chunks and add them to the prompt, so answers rest on correct theory and cite sources.

Why TF-IDF and not only embeddings?

Works offline with no model download; Sentence Transformers can be switched on.

What if the LLM is wrong?

Numbers come from the database, not the LLM; the prompt restricts it to the given data with citations; the Copilot only advises; offline mode is deterministic.

What about HTTPS traffic?

Payload is encrypted; we use headers, ports, sizes and timing only.

Why is the demo synthetic?

So the truth is known: we inject problems at known times and check NetMind finds them. Real captures and NS2 traces go through exactly the same pipeline.

Why was delay lower with RED?

RED (Random Early Detection) drops packets at random once the average queue passes a threshold, before the queue is full, so TCP backs off earlier and the queue stays short. In our run delay during congestion fell from 103.1 to 71.3 ms, with 25 drops instead of 21.

What is the health score?

Each 2 s window starts at 100 and loses up to 40 points for latency above normal, up to 40 for loss and 20 if flagged as anomalous. Demo 82.4 (Fair), normal traffic 99.5 (Good).

How do you find a port scan without the ML model?

By following each TCP SYN: 192.168.1.66 made 1,000 attempts to 1,000 ports, all refused with RST. 20+ failed attempts to 20+ ports marks a likely scanner.

Ethics?

Only capture networks you own or may monitor; the Copilot never changes the network.

11. Limitations and next steps

- Demo data is synthetic; thresholds were tuned on it and should be checked on real captures.
- Topology is logical (who talked to whom), not physical cabling.
- Passive loss is an estimate (retransmissions and unanswered pings).
- **Next:** analyse real campus/home captures; more NS2 scenarios (more nodes, other queue types); evaluate the Copilot answers; record the NS2 results and NAM screenshots in ns2/results.

12. Changes since Review 1

- NS2 Simulation Module added, as the faculty suggested; now executed in NS2 with all checks passing.
- Full implementation of all 9 modules, CLI, REST API and dashboard; automated tests.
- Fewer false alarms; honest "not enough traffic" instead of "no anomalies" for short captures.
- Redesigned dashboard with 10 pages (including an NS2 lab page), Copilot panel, link-failure simulator, light and dark themes.
- New analysis: TCP connection outcomes, DNS insights, host inventory with roles and services, and a 0-100 health score; NS2 scenario also run with a RED queue: 31% lower delay during congestion.

Appendix A. How to run everything

Windows (PowerShell), in the project folder

Every time you open a new PowerShell window:

```
cd C:\Users\moham\NetMind
.\.venv\Scripts\Activate.ps1          # the prompt starts with (.venv)
# if scripts are blocked:  Set-ExecutionPolicy -Scope Process Bypass   then run the line above again
```

Once, or after every git pull:

```
git pull
python -m pip install -r requirements.txt
python -m pytest -q                       # all tests; note the passed count
```

Commands

What	Command	What you should see
Synthetic demo	python -m netmind demo	Per-window table and 3 incidents (congestion, port scan, flood)
Dashboard	python -m netmind dashboard	Browser opens http://localhost:8501; keep this window open
Ask the Copilot	python -m netmind ask "Why is the network slow?"	Answer naming the congestion at 70-100 s, with sources
See the prompt	python -m netmind ask "Is anyone attacking my network?" --show-prompt	The full prompt: data, incidents, retrieved knowledge
Routing	python -m netmind routes --from 192.168.1.10 --to 8.8.8.8	Dijkstra path; Link State vs Distance Vector tables
Routing by hops	python -m netmind routes --from 192.168.1.10 --to 8.8.8.8 --weight hops	Same, every link costs 1
Analyse the NS2 trace	python -m netmind ns2 ns2\results\netmind.tr	Congestion 3-6 s and link failure / reroute via n4
Hosts, connections, DNS	python -m netmind hosts	9 hosts with roles; 1,363 TCP attempts, 1,000 refused from 192.168.1.66; 364 DNS lookups; health 82.4/100
List runs / report	python -m netmind runs python -m netmind report --run 1	Stored runs; metrics and incidents of one run
REST API	python -m netmind api	http://127.0.0.1:8000/docs (interactive API page)
Live capture (optional)	python -m netmind capture --duration 60	Needs Npcap and PowerShell run as Administrator
Help	python -m netmind --help	All commands

In the dashboard: sidebar **Run** box picks the run; **Start a new run → Generate and analyse** makes a demo run; **Analyse file** loads ns2\results\netmind.tr as an NS2 run. Stop the dashboard with Ctrl + C in its PowerShell window.

Ubuntu (NS2)

Open the Ubuntu terminal (Start menu → Ubuntu). Paste with Ctrl + Shift + V.

```
cd ~/netmind_ns2
ns netmind_scenario.tcl 2>&1 | tee ns_run_output.txt   # runs the 10 s simulation
python3 check_trace.py netmind.tr                      # 5 checks, all PASS
nam netmind.nam                                        # animation (see Appendix B)
cp netmind.tr ns_run_output.txt /mnt/c/Users/moham/NetMind/ns2/results/   # trace to Windows
```

Appendix B. Showing NS2 starting cleanly (no errors on screen)

Do steps 1 and 2 once before the review day; do step 3 in front of the panel.

1. Fix the two warnings we saw (once)

What you saw	Cause	Fix
"ns found the right version of tclsh in /bin/tclsh8.6 but it doesn't seem to be there anymore"	Tcl 8.6 was not installed	`sudo apt install -y tcl8.6 tk8.6` (already done on 15 Oct). Check: type `ns`, you get a `%` prompt with no warning; type `exit`
NAM prints only "nam:" and quits	NAM cannot find the Tk library file tk.tcl (seen with strace)	Find it: `find / -name tk.tcl 2>/dev/null`; then set TCL_LIBRARY and TK_LIBRARY to its folders (step 2). If find prints nothing: `sudo apt install -y libtk8.6 tk8.6-dev tcl8.6-dev` and find again
"ns -version" opens a % prompt	ns2 has no -version option	Not an error. Type exit. For the version: in the % prompt type `puts [ns-version]`

2. Make the NAM fix permanent (once, after NAM has opened)

Use the folders that **find** printed (the folder containing tk.tcl for TK_LIBRARY, and the matching tcl8.6 folder for TCL_LIBRARY). If it printed /usr/share/tcltk/tk8.6/tk.tcl:

```
echo 'export TCL_LIBRARY=/usr/share/tcltk/tcl8.6' >> ~/.bashrc
echo 'export TK_LIBRARY=/usr/share/tcltk/tk8.6' >> ~/.bashrc
source ~/.bashrc
```

3. In front of the panel

```
cd ~/netmind_ns2
clear                                   # start from an empty screen
ns netmind_scenario.tcl demo_run        # writes demo_run.tr and demo_run.nam
```

Expected output, and nothing else:

```
NetMind NS2 scenario: running 10 s simulation, output base name 'demo_run', bottleneck queue DropTail
t= 0.50 s  TCP flow n0 -> n3 starts
t= 3.00 s  UDP burst n1 -> n3 starts (congestion expected)
t= 6.00 s  UDP burst stops
t= 7.00 s  link n2-n3 down (DV should reroute via n4)
t= 8.50 s  link n2-n3 up
Simulation finished at t=10 s
Trace file:     demo_run.tr
NAM animation:  demo_run.nam   (open with: nam demo_run.nam)
```

Then:

```
python3 check_trace.py demo_run.tr      # 5 x [PASS]
nam demo_run.nam                        # NAM window opens
```

In NAM: press ▶ (play), slow it down with the **Step** slider, and pause at **3 s** (UDP burst fills the n2-n3 queue, packets drop), **7 s** (n2-n3 link down, TCP goes n2 → n4 → n3) and **8.5 s** (link back up). The time is shown at the top of the NAM window.

Rehearse this once the day before. If NAM still does not open, show the NAM screenshots or the NS2 run in the dashboard; the simulation and the checks are what prove NS2 works.